

AgroVision

10 - Backend / Infra

FastAPI, Uvicorn, Railway e Roboflow Serverless

TCC-Docs · Material de estudo
Luiz Carlos Junior & Equipe AgroVision
Maio de 2026

1. Stack do backend

Camada	Tecnologia	Versao
Framework web	FastAPI	0.110
Servidor ASGI	Uvicorn (com uvloop)	0.29
ML runtime	TensorFlow (CPU)	2.20.0
Keras	Keras 3 nativo	3.13.2
Visao computacional	OpenCV (headless)	4.8
Numerica	NumPy	1.26
HTTP client	requests	2.31
Imagem	Pillow	10.1
Python runtime	CPython	3.12

2. Por que FastAPI

FastAPI foi escolhida sobre Flask pelo suporte nativo a **async/await**, **WebSocket** (o Grad-CAM precisa de progresso em tempo real) e **validacao automatica de schemas** via Pydantic. Tambem gera automaticamente documentacao Swagger em /docs, util durante o desenvolvimento.

3. Endpoints

Rota	Tipo	Funcao
/api/health	GET	Healthcheck + metadata do modelo
/ws/classify	WS	Classificacao (Roboflow) com progresso
/ws/gradcam	WS	Grad-CAM + AL/AFS com progresso
/api/xai/gradcam	POST	Mesma funcao do WS, multipart/form-data (fallback)

4. Fluxo de uma requisicao WebSocket

1. Cliente abre WS com ?api_key=...
2. Backend valida a chave. Se invalida, fecha com codigo 4003.
3. Cliente envia primeira mensagem JSON com image_b64 e parametros.
4. Backend emite eventos *{preprocessing, classifying / building_model, computing_gradients, rendering_overlay, done}* com pct.
5. Backend executa a operacao em threadpool (TF nao e async-friendly).
6. Backend emite *{result, data}* com o payload final.
7. Fecha o WS.

5. Autenticacao

Header X-API-Key (REST) ou query param ?api_key=... (WS). A chave esperada vem da variavel de ambiente AGROVISION_API_KEY no Railway. Se a variavel nao estiver setada (ambiente local), o backend roda sem autenticacao.

6. Roboflow Serverless

O backend funciona como **proxy** para o Roboflow Serverless quando ROBOFLOW_API_KEY e ROBOFLOW_MODEL_ID estao configurados. Fluxo interno em _classify_sync:

1. Recebe bytes da imagem.
2. Redimensiona para 640px no maior lado (evita erro 413 de payload muito grande).
3. Encode em base64.
4. POST para `https://serverless.roboflow.com/{slug}/{version}?api_key=...`
5. Parse do JSON, extrai top-1 prediction.
6. Retorna {prediction, confidence, source: 'roboflow'}.

7. Deployment no Railway

Build system: Nixpacks (deteccao automatica de Python pelos arquivos requirements.txt + runtime.txt).

Arquivos de config:

- Procfile — comando de start
- runtime.txt — python-3.12
- nixpacks.toml — fases customizadas de build
- requirements.txt — dependencias fixadas

Plano: Hobby (US\$ 5/mes), inclui ~US\$ 5 de uso. Para um TCC com baixo trafego e suficiente.

8. Variaveis de ambiente

Variavel	Funcao
ROBOFLOW_API_KEY	Chave para a API serverless
ROBOFLOW_MODEL_ID	Slug do modelo no formato workspace/slug/version
AGROVISION_API_KEY	Chave compartilhada entre app e backend
PORT	Setada automaticamente pelo Railway

9. Limites e observacoes operacionais

- **Cold start:** primeira requisicao apos inatividade demora ~10s (TF carrega o modelo).

- **Memoria:** TF 2.20 + Keras + modelo carregado consomem ~800 MB — cabe confortavelmente no plano Hobby (8 GB).
- **CPU:** uma inferencia + Grad-CAM gasta ~300-450 ms em CPU. Sem GPU, mas suficiente para uso interativo.
- **Logs:** stdout via Uvicorn, visível no dashboard do Railway em tempo real.